

WClouds Doku POS

Team

Jakob Wörz und Karan Özcelik

1. Projektbeschreibung

Was ist WClouds?

WClouds ist ein Cloud basiertes Speichersystem das Clients erlaubt kostenlos 100 Gigabyte Speicher zu haben. Wir machen dies mit einem alten Laptop den wir zu einem Art Home-Server bauen. Mithilfe einer Benutzer Datenbank erlauben wir nur bestimmte Kunden. Registrierung erfolgt per Email Passwort und einer Datei die als Key dient (Wäre uncool einen HomeServer für jeden frei zu machen) so können nur Leute die auch den Key besitzen unsere Cloud verwenden

2. Planung

Must-Have:

- Anmeldung/Registrierung
- Managen von Daten
- Accounts/Daten Deleten
- Nach langer Innaktivität Account und Daten löschen
- Alles verschlüsselt von Cloud Daten bis zu User Daten in der database
- HTTPS statt HTTP
- File Sharing

Nice-Have

- Animiertes Frontend/ verschönertes Frontend (Anfangs vielleicht weniger fokussieren)
- File Explorer View (Daten managebar im File Explorer)
- Login Timeouts
- Email Warnung bei langer Innaktivität

GUI-Konzept

Fensterübersicht

1. LoginPage: Sich mit **Email** und **Passwort** anmelden.
 2. RegisterPage: Sich mit **Email**, **Passwort** und **Storage Key** registrieren.
 3. DataPage: Hier werden alle gespeicherten und gesendeten Files und Folder angezeigt.
-

3. Umsetzung

3.1 Verwendete Technologien

Technologie / Paket	Version
.NET	10.0
WPF	net7.0-windows
pwdlib(argon2)	0.3.0
python-multipart	0.0.20
System.Security.Cryptography	10.0.0.0
Serilog	4.3.1

3.2 Projektstruktur

Das System besteht aus mehreren unabhängigen Funktionsblöcken.

Benutzerverwaltung

Verantwortlich für Registrierung und Anmeldung von Benutzern.

Klassen:

- Authenticator
- User
- Webservice

Funktionen:

- Benutzer registrieren
- Passwort-Hashing
- Login
- Session-Key Verwaltung

Dateiverwaltung

Verantwortlich für das Speichern und Abrufen von Dateien und Ordnern.

Klassen:

- StorageService
- SavedFile
- SavedDirectory

Funktionen:

- Datei hochladen
- Datei herunterladen
- Ordner hochladen
- Ordner herunterladen
- Dateiinformationen abrufen

Verschlüsselung

Verantwortlich für die lokale Verschlüsselung der Daten.

Klassen:

- EncryptionService

Funktionen:

- AES-256-GCM Verschlüsselung
- AES-256-GCM Entschlüsselung
- Schlüsselverwaltung mittels Windows DPAPI

Freigaben (Sharing)

Verantwortlich für das Teilen von Dateien zwischen Benutzern.

Klassen:

- ShareService
- SharedFile
- FileAccess

Funktionen:

- Datei freigeben
- Rechte vergeben
- Rechte entziehen
- Freigegebene Dateien anzeigen

Backupverwaltung

Verantwortlich für den Zugriff auf Sicherungen.

Klassen:

- BackupService

Funktionen:

- Backup-Dateien abrufen
- Backup-Ordner abrufen

Benutzeroberfläche

Verantwortlich für die Bedienung des Systems.

Klassen:

- MainWindow
- SignInPage
- RegisterPage
- DataPage
- ShareDialog

Funktionen:

- Navigation
- Dateiverwaltung
- Anzeige der Ordnerstruktur
- Statusmeldungen

3.3 Detaillierte Beschreibung der Umsetzung

Registrierung

Der Benutzer gibt E-Mail-Adresse, Passwort und einen Storage-Plan-Key ein.

Ablauf:

1. Passwort wird mittels SHA-256 gehasht.
2. Storage-Key wird Base64-kodiert.
3. Daten werden an die REST-API gesendet.
4. Der Server erstellt den Benutzer und den Root-Ordner.

Anmeldung

1. Benutzer gibt E-Mail und Passwort ein.
2. Passwort wird gehasht.
3. Login-Anfrage wird an den Server gesendet.
4. Der Server liefert einen Session-Key zurück.
5. Der Session-Key wird im HttpClient gespeichert.

Datei-Overwrite

1. Benutzer wählt Datei aus.
2. File Explorer öffnet sich und man muss seine gewünschte Datei aussuchen die jeweilige ersetzt.
3. Inhalt der Datei wird überschrieben und in die History abgespeichert.

Datei-Upload

1. Benutzer wählt eine Datei aus.
2. Datei wird lokal eingelesen.
3. EncryptionService verschlüsselt die Datei mittels AES-256-GCM.
4. Verschlüsselte Daten und Nonce werden an den Server übertragen.
5. Der Server speichert ausschließlich die verschlüsselten Daten.

Datei-Delete

1. Benutzer wählt Datei aus.
2. Datei wird mit zugehöriger ID verglichen.
3. Datei wird aus der Datenbank gelöscht.

Datei-Download

1. Benutzer fordert eine Datei an.
2. Server liefert verschlüsselte Datei und Nonce.
3. Client entschlüsselt die Datei lokal.
4. Datei wird gespeichert.

Datei-History

1. Benutzer wählt Datei aus.
2. Falls die Datei bereits überschrieben wurde werden die alten Versionen angezeigt.
3. Diese kann man als Backup runterladen.

Ordner-Upload

1. Ordnerstruktur wird rekursiv durchlaufen.
2. Für jede Datei wird UploadFile aufgerufen.
3. Unterordner werden rekursiv verarbeitet.
4. Die Struktur wird auf dem Server nachgebildet.

Ordner-Download

1. Server liefert ein ZIP-Archiv.
2. Archiv enthält verschlüsselte Dateien und zugehörige Nonces.
3. Dateien werden lokal entschlüsselt.
4. Ursprüngliche Ordnerstruktur wird wiederhergestellt.

Dateifreigabe

1. Besitzer wählt eine Datei aus.
2. Benutzer-ID des Empfängers wird ermittelt.
3. Lese- und Schreibrechte werden definiert.
4. Freigabe wird auf dem Server gespeichert.
5. Datei erscheint beim Empfänger unter „Geteilt mit mir“.

3.4 Mögliche Probleme und unsere Lösung

Problem	Lösung
Dateien sollen sicher gespeichert werden	Die Dateien werden vor dem Hochladen verschlüsselt. Dadurch kann niemand die Inhalte auf dem Server lesen.
Nur berechtigte Benutzer sollen auf Dateien zugreifen können	Beim Teilen einer Datei werden Lese- und Schreibrechte festgelegt. Der Server überprüft diese Rechte bei jedem Zugriff.
Der Verschlüsselungsschlüssel darf nicht verloren gehen	Der Schlüssel wird sicher auf dem Computer des Benutzers gespeichert und beim nächsten Start der Anwendung wieder verwendet.

4. KI-Unterstützung

Verwendete Tools

- ChatGPT
- Claude

5. Projekttagebuch

Datum	Person	Tätigkeit
21.05.2026	Wörz	Initial Commit
28.05.2026	Wörz	Alle Basisklassen und beim Authentifizieren vieles geschafft
28.05.2026	Karan	Backend eingerichtet
31.05.2026	Wörz	SignInPage und RegisterPage (fertig) gemacht
02.06.2026	Wörz	Meisten Klassen fertig und Einloggen/Registrieren erfolgreich
03.06.2026	Wörz	Neue Login Einstellungen
08.06.2026	Karan	En- und Decryption
10.06.2026	Karan	GUI verbessern
10.06.2026	Wörz	Backend eingefügt, KI Kommentare, logout und zurück buttons
11.06.2026	Karan	xUnitTests und Share Service

Datum	Person	Tätigkeit
11.06.2026	Wörz	Folder runterladen
15.06.2026	Wörz	ShareService gefixed, canread/canwrite eingebaut und alle Methoden fertig gemacht
16.06.2026	Wörz	Restlichen KI Kommentare
17.06.2026	Wörz	Logging eingebaut
18.06.2026	Karan	WCloud File Explorer Integration eingebaut, Infos anzeigen lassen, Delete

6. Reflexion

Was lief gut?

- Zeitplan
- Verschlüsseln

Was lief schlecht?

- Wenig Lust teilweise
- Bisschen wenig committed

Wo war KI hilfreich?

- UI Design
- Sicherheitslücken entdecken

Was würden wir anders machen?

- Aufgabenverteilung bei der Planung
 - Realistischere Ziele
-

7. Repository

GitHub-Link: <https://github.com/JW-2704/WClouds>